Implementing BUBBLE-BLUE over BLE L2CAP: An Open Mesh with Private Groups

Paul-Maxence Baraton, Nadjib Achir, Philippe Jacquet
INRIA, France
{paul-maxence.baraton, nadjib.achir, philippe.jacquet}@inria.fr

Wilfred Doré
Qualcomm, France
wdore@qti.qualcomm.com

***Abstract*—BUBBLE-BLUE (BB) is a multi-hop private group that rides on an *open* Bluetooth mesh: any BB node may forward, only holders of a group key may read. Prior work specified the construction and evaluated Connected Dominating Set (CDS) election in simulation. This paper reports the first implementation, on BLE L2CAP Connection-Oriented Channels, without Bluetooth pairing. We describe the on-wire format, the split between a topological plane and an application plane, and a repeatable evaluation on Qualcomm Linux boards (RUBIK Pi 3 and Arduino UNO Q). A three-node mesh completes in 7–14 s, remains complete for 97–98% of a steady run, and delivers 100% of chat. Driving the shipped forwarder in simulation, the smallest TTL that reaches every node equals the source's hop eccentricity: the CDS adds no path stretch. The practical limit is therefore a hop budget (default TTL 10 spans about twenty nodes at mean degree 4–5). AES-256-GCM runs at 30k–173k packets/s, far above the BLE link; the Arduino host stack saturates near 10 messages/s. Revocation excludes the victim in every trial and strands legitimate members only inside the join window, before public keys have flooded.**

***Index Terms*—Bluetooth Low Energy, L2CAP, mesh networks, connected dominating set, group communication.**

I. Introduction

Infrastructure-less local communication still collapses, in practice, onto a Wi-Fi access point or a smartphone OS that will not host a third-party mesh. Bluetooth is ubiquitous and needs no operator. The SIG Mesh profile [1], however, is a different stack, designed for managed flooding (lighting, sensors), not for a private group that forms between people who share a key and nothing else.

BUBBLE-BLUE [2] proposed the opposite split: an *open topological mesh* that any BB node may join and relay, and *private application bubbles* readable only with a group key. Routing is a dynamic CDS. That note gives the algorithms and unit-disk simulations. It does not give a BLE transport, a frame layout, a revocation protocol that reaches two-hop members, or measurements on hardware.

This paper fills that gap. The implementation runs on two Qualcomm Linux boards over BLE L2CAP CoC [3]. Connections use `BT_SECURITY_LOW`: there is no SMP pairing, no bond, and no address allow-list. Membership is the group key. A node that has never met another can still talk to it. A passive sniffer in range can read routing headers; it cannot read payloads (Section III-E).

Our contributions are:

1) A complete BB stack on BLE CoC, including the 26-byte routing header, packet types, opaque forwarding, and epoch-based revocation (Section III).
2) A repeatable multi-board evaluation method and the figures it produces on two platforms (Section V).
3) Characterised operating limits: hop-TTL, join-window revocation, and per-platform saturation—including two implementation defects the limit studies exposed and that are now fixed (Section V-D).

The rest of this paper is organised as follows. Section II positions BB against CDS literature and existing BLE messengers. Section III presents the design. Section V evaluates the implementation. Section VI concludes.

II. Related work

a) CDS and multi-hop broadcast.: Wu and Li's 1999 local rule [4] is our default election; their 2001 scheme [5] is the broader dominating-set routing setting. Multipoint relays [6] are the OLSR construction [7]; in BB, MPR is an alternative *algorithm* whose *result* is still a CDS. Jacquet's analytic densities [8] and the 1D/2D unit-disk comparison in [2] are why Wu–Li was preferred for a first implementation: simpler, and better than MPR on the 2D disk in that note. Dumbo II [9] is the emergency-network precedent cited in [2]. BLE has no native IP-style broadcast: every local flood is a sequence of L2CAP unicasts. That cost model is why a CDS (fewer transmitters) matters more here than on Wi-Fi MANETs.

b) BLE transports.: We use LE Connection-Oriented Channels [3], not RFCOMM (Classic, serial, no BLE equivalent) and not GATT as the data plane (serialised writes, no native fan-out to CDS neighbours). Advertising carries *presence* only. SIG Mesh [1] remains a separate, provisioned stack.

c) Adjacent messengers.: BitChat [10] shows that a BLE-only chat can exist at consumer scale; it floods over GATT and elects no backbone. Briar [11] is delay-tolerant sync between *pre-paired* contacts over Classic RFCOMM, Wi-Fi and Tor—a different problem. Reticulum [12] is the closest Linux BLE stack; it is identity-routed, not CDS-routed. Bridgefy and BLEMeshChat likewise flood or store-and-forward without a CDS. None of these is a BB node. Netcore Causeway [13] shares high-level aims with different technology; we do not treat it as a baseline.

TABLE I
WHAT BB IS NOT. "OPEN RELAY" MEANS A NODE MAY FORWARD A FRAME IT CANNOT DECRYPT.

System	Data plane	Backbone	Open relay
SIG Mesh [1]	managed flood	provisioned	no
BitChat [10]	GATT	none	n/a
Briar [11]	RFCOMM, Wi-Fi, Tor	contacts	no
Reticulum [12]	multi-transport	identity	no
BUBBLE-BLUE	LE CoC	dynamic CDS	yes

Table I is the distinction we need in front of a systems audience. SIG Mesh is a product profile with its own provisioning. BitChat shows that a phone can chat over BLE alone, by flooding GATT writes. Briar synchronises mailboxes between people who have already paired. Reticulum routes to cryptographic identities across whatever link is up. BB is the remaining cell: a CDS elected on an open BLE mesh, with confidentiality only inside a group that shares a key. A node can sit in the CDS and relay for a bubble it does not belong to. None of the other four rows does that.

III. DESIGN

A BB node is a Linux process that owns an HCI adapter, accepts CoC connections on PSM `0x0081`, and runs the control and forwarding logic below. Pairing is unused. Confidentiality is application-layer.

A. Open mesh, private bubble

Two planes share the same links and the same 26-byte header, and nothing else.

The **topological plane** is open. Any BB node may advertise presence, accept a connection, take part in CDS election, and relay a frame it cannot decrypt. AES-GCM failure is logged; it is never a reason to drop the relay.

The **application plane** is private. Group traffic, key announcement and revocation are readable only with the current group key. Membership is that 32-byte secret, not a Bluetooth bond.

This is the construction of [2], realised as opaque forwarding of group data by nodes that may not belong to the bubble.

B. Presence and sparse links

A node advertises a compact presence record in a connectable legacy advertisement (manufacturer data, magic `BB`): anonymous identifier, CDS flag, and the BD_ADDR at which it accepts CoC. Scanners accumulate a *presence catalog* indexed by anon_id. The catalog survives a dropped socket and is how a neighbour is redialled without pairing.

Links are not a full mesh. Each node caps its L2CAP degree and prefers CDS successors. On a six-node soak, boards held two or three links each and still reached the other five. When the CDS is empty—at start-up, and whenever legacy advertisements cannot carry neighbour lists under a $\sim$27-byte budget—a node dials from the catalog rather than waiting for an air-only election that cannot see two-hop topology.

The implementation runs on RUBIK Pi 3 (Ubuntu, Bluetooth 5.0) and Arduino UNO Q (Debian plus an STM32, Bluetooth 5.1). CPython 3.14 is required for LE 4-tuple sockets. A privileged daemon owns the adapter; the UI is an unprivileged client on an `AF_UNIX` socket, so restarting a display does not drop links. The same split is what lets a panel be a spectator: it draws peers, hops and revocation state, and it never holds a CoC.

The compact advertisement is 23 bytes of manufacturer value before any neighbour list: magic and version (4), anonymous identifier (8), sequence (4), flags (1), reachable BD_ADDR (6). Legacy advertising accepts about 27 bytes of manufacturer data in total, so the neighbour and MPR lists that the original note puts on the air do not fit. They are carried later, in HELLO, once a CoC exists. A scanner that sees only the advertisement can already dial: the BD_ADDR in the value is the address that accepts PSM `0x0081`. It cannot be taken from the advertising address itself, because a non-connectable advert may rotate that address.

Joining is then four steps, and they are not the same timeout. Presence is one advertising interval. The CoC dial is one L2CAP handshake. HELLO fills the neighbour table and starts the miss timer. ANNOUNCE, on a 30 s period, is the step that delivers public keys beyond one hop and thereby closes the revocation window of Section III-E. A spectator who taps "send" during the first of these steps is early; the mesh is up, the bubble is not yet able to exclude anyone safely. The panels therefore show link state separately from group epoch.

C. Routing header and packet types

Every frame begins with a 26-byte cleartext header:

```
type(1) | src(6) | dst(6) | ttl(1) | seq(4
              BE) | ts_ms(8 BE)
```

Relays decrement TTL on the wire, so TTL is *not* in the AES-GCM additional authenticated data; the rest of the header is. A relay without the group key can still forward, and cannot forge an AAD-covered field. Broadcast destination is `FF:FF:FF:FF:FF:FF`.

The ciphertext is AES-256-GCM [14], [15] with a 16-byte tag. There is no nonce on the wire: the per-packet key is derived from the group or pairwise secret, source, sequence and timestamp. Overhead is 42 bytes. Against a conservative 512-byte MTU that leaves 470 bytes of plaintext. Both boards negotiate L2CAP MTU 672, so a maximum-size message is still one frame; 512 is policy. A 471-byte payload is refused, never truncated.

Two sequence counters number these frames (Hello/ANNOUNCE versus DATA/REKEY). They must not share an anti-replay window (Section V-D).

D. CDS, forwarding, and membership expiry

Roles follow Wu–Li 1999 [4]. MPR [6] is implemented in the simulator; boards run Wu–Li. A node forwards if it is RELAY, or if it has two or more L2CAP links (a bridge the CDS has not named—the usual case at the edge of range,

TABLE II
BB PACKET TYPES.

Type	Name	Plane	Role
0x01	HELLO	both	Neighbour table, epoch, CDS
0x02	DATA	private	Group broadcast
0x04	SUBGROUP	private	Addressed multicast
0x05	REKEY	private	Pairwise epoch catch-up
0x06	ANNOUNCE	private	Signed pubkey binding
0x07	REKEY_FLOOD	private	Coordinator revocation

where the advertisement graph is a clique and the L2CAP graph is a star). Duplicate suppression is keyed by (src, seq) and by which counter produced seq.

A neighbour is dropped after HELLO_MISSES consecutive silent Hello periods (default 3, minimum 2), i.e. 15 s at a 5 s interval. Counting periods keeps the threshold meaningful if the interval is retuned.

E. Security and revocation

Link encryption is off. Group AEAD is AES-256-GCM under the group key; pairwise keys are X25519 [16] after ANNOUNCE; rekeys are attested with Ed25519 [17]. The routing header is visible to a sniffer; payloads are not.

A passive sniffer in range therefore learns who transmitted, toward which broadcast or unicast address, and how far the frame may still travel. It does not learn the text, the group key, or the pairwise secrets. It also cannot flip TTL and have the frame accepted as authentic: TTL is the one header field excluded from the additional authenticated data, precisely so a relay can change it, and every other header field is covered. An active attacker who injects a frame still has to produce a valid tag under a key they do not hold. What this does *not* give is anonymity of the radio address: BD_ADDR is in the header and on the advertisement, on purpose, because that is how a neighbour is dialled without a bond. Anonymous identifiers hide the group membership of a presence record; they do not hide the controller.

The coordinator is the member with the smallest anon_id among *all known members* (Hello-resolved *and* ANNOUNCE-learned). Election and validation of a rekey must use that same set (Section V-D).

Only the coordinator may revoke. It bumps the epoch, rotates the group key, and floods REKEY_FLOOD wrapped under *pre-bump* pairwise keys, excluding the victim. Late members catch up on the next stale Hello. The coordinator cannot wrap a key it does not hold: Hello supplies direct neighbours only; everyone else arrives in ANNOUNCE (30 s). Revoking inside that *join window* is refused or strands legitimate members—an operating limit, not a demonstration surprise.

Both SoCs use ARMv8 Cryptography Extensions (gcm-aes-ce via OpenSSL). Qualcomm QCE is present but does not register GCM; it is not on the path.

F. One group message, hop by hop

Consider a chat line sent by screen A toward screen B, with a UNO Q relay R between them and no direct A–B link. A builds a DATA frame: broadcast destination, TTL set to the configured maximum, sequence taken from the data counter, timestamp in milliseconds. The payload is the UTF-8 text. The per-packet key is derived from the current group key, A's address, the sequence and the timestamp; AES-GCM seals it. The 26-byte header stays clear.

A writes that frame on every CoC it holds. R accepts it, checks the relay anti-replay window for (A, seq) in the data counter's space, decrements TTL, and writes the same ciphertext to its other peers. R does not need the group key to do this. It then tries local delivery: GCM fails if R is not a member, and the frame is still forwarded. B decrypts, checks its own delivery window, and hands the text to the local UI. A copy that returns to A is dropped because the source is local. A second copy that arrives at B through another relay hits the same (src, seq) window and is dropped.

Two consequences follow. First, adding a relay does not require that relay to hold the group key: the open mesh and the private bubble are separated on the wire, not only in the specification. Second, the UI can show the ingress neighbour and a hop count ($ttl_{\max} - ttl + 1$) without reading any node beyond its own daemon. That is what a spectator sees move on the screen.

G. What the UNO Q controller will not do

The two boards do not present the same HCI behaviour, and the mesh only stays up if the implementation respects both.

a) Central-only links.: The UNO Q's WCN399x firmware is driven by hci_qca. That driver does not advertise valid LE states, so the kernel sets HCI_QUIRK_BROKEN_LE_STATES. While the adapter is peripheral on any link, check_pending_le_conn() never sends an LE Create Connection. A dial then times out with no connection attempt on the air. The RUBIK Pi 3 uses a Broadcom controller and does not have this quirk. On a stand where the screens must be reached through the relays, each UNO Q is therefore the central of every link it holds: it advertises non-connectable and dials, and each RUBIK advertises connectable and never dials. With two screens and three relays the steady graph is the complete bipartite graph $K_{2,3}$: three independent two-hop paths, and no screen–screen or relay–relay edge is required for the chat.

b) Signalling-identifier leak.: Each received LE credit on these kernels consumes one L2CAP signalling identifier and does not return it. The pool is 254 identifiers per ACL. Under a quiet chat this is invisible; under the topology traffic of a five-node mesh a link exhausts the pool in minutes and the kernel logs Unable to allocate ident. The daemon recycles a link after 200 received credits: it drops the CoC and disconnects at the MGMT layer, then dials again. Neighbours treat this as an ordinary flap. The alternative, restarting BlueZ or the board, is what a day-long demonstration cannot afford.

A third failure is less regular. The WCN399x UART has stopped answering HCI commands after a few hours of mesh traffic, with transmit timeouts and a firmware version read of -110. Rebinding the HCI UART driver reloads the firmware

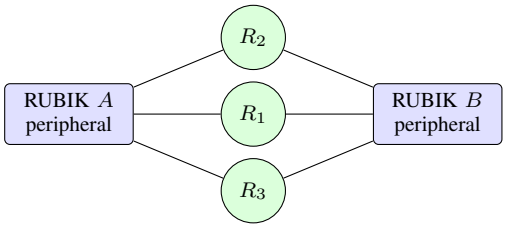


Fig. 1. Demonstration topology. Each UNO Q relay is central toward both screens. A spectator message crosses one of the three two-hop paths; the screen draws that path as the text arrives.

and restores a powered adapter in tens of seconds, without rebooting the board. Each daemon watches its own controller and does this locally: a restart order flooded on the mesh cannot reach a radio that is already dead.

IV. A FIVE-NODE STAND

Figure 1 is the configuration we run in front of an audience. Two RUBIK Pi 3 boards carry touch panels. Three Arduino UNO Q boards are relays and have no display. The panels show preset sentences rather than a keyboard, the live $K_{2,3}$ beside the transcript, and, on one screen only, an operator action that revokes a chosen member.

Revocation on that stand is the protocol of Section III-E, not a second mechanism. The operator names a board; the coordinator omits that board from `REKEY_FLOOD`; the next preset sentence is readable on every remaining member and not on the victim. The victim's panel says it has been excluded, which is the point of the demonstration: the mesh is still there, the bubble is not. An explicit operator reset returns every board, including the victim, to the bootstrap epoch-0 key so the same table can be shown again. That reset is a demonstration control. It is not part of the group protocol, and a revoked member cannot invoke it.

The numbers in Section V predate this five-node arrangement. They are the controlled runs we already have (formation, departure, TTL, revocation cost, host-stack saturation). A campaign on the stand itself—join delay, delivery over a full day, recovery after a forced controller stall—is the next measurement, not a result of this draft.

V. EVALUATION

Experiments use simultaneous probes on all boards, the same git commit, and *worst-board* figures (a mesh is complete when the last node has joined). Warm-up (25 s) is counted as convergence, not as instability. Departure is timed on the presence table, not on L2CAP sockets (a socket dies with its process regardless of `HELLO_MISSES`). Load tests use a single sender; each message carries an index, and delivery is that index seen by a listener. Simulation instantiates the shipped forwarder on a unit-disk radio and drains floods breadth-first (call-stack, depth-first delivery is not a radio).

A. Convergence, stability, and departure

The two losses in 429 were the first message (before convergence) and the last (during shutdown). Applications should display mesh state rather than assume delivery in the first tens of seconds. Relay role flips in the automated runs: 0. Steady-state L2CAP flaps: a few per run, all recovered.

TABLE III
MESH FORMATION AND CHAT DELIVERY.

Setting	n	Full mesh	Steady complete	Delivery
Automated	3	7–14 s	97–98%	100%
Automated	2	3–14 s	97%	100%
Manual soak	6	∼40 s	stable after 60 s	427/429

TABLE IV
DEPARTURE DELAY VS. CONFIGURED HELLO MISSES (PRESENCE VIEW).

`HELLO_MISSES`	Expected	Measured departure	Rejoin
3	15 s	17–21 s	7–12 s
6	30 s	32–36 s	7–10 s

Killing a node without notice (Table IV): departure tracks the configured miss count. Rejoin does not: it is discovery and dial.

B. Crypto is not the bottleneck

BLE CoC carries a few hundred packets per second. AES-GCM is two to three orders of magnitude above the link (Table V). Energy of the crypto path is not measured.

C. Topological and revocation limits

At mean degree ∼4–5, the minimum TTL that delivers to every node equals the source eccentricity on every draw (chains and random geometric graphs):

There is no "it breaks at N nodes". Past a 10-hop diameter, default TTL 10 no longer spans the network—about twenty nodes at this density. At six nodes the default is ample. Below mean degree ∼5, no connected geometric draw in 100 trials (percolation). Denser graphs elect fewer relays (52%→38%) and cost more per flood.

After ANNOUNCE has run, revocation at $n = 6$–30: 0 stranded members, 0 leaks to the victim. Cost grows from 12 to 83 emissions per node (roughly quadratic). Before ANNOUNCE, revocation is often refused. Rule: wait one ANNOUNCE period after a join.

D. Load and two defects the study caught

A single sender, 64–470 byte payloads. The RUBIK listener received 100% through 30 msg/s. The UNO Q tracked to 10 msg/s, lost at 15, then delivered 0—including a 1 msg/s recovery step after 25 s idle—while its links stayed nominally up. The kernel logged `Unable to allocate ident: -28` (`ENOSPC` on BlueZ L2CAP *signalling* identifiers); controller error counters stayed 0. Data-plane saturation delays Hellos, neighbours expire, auto-dial bursts, the ident pool empties. Restarting the Bluetooth service recovers the board. A separate QCA firmware stall after ∼6 weeks of uptime requires a reboot; it is not this failure.

Limit studies also found two protocol bugs, both now fixed.

TABLE V
AES-256-GCM ON A 470-BYTE PAYLOAD (MEAN OF 20 000 CALLS).

Platform	Encrypt + decrypt	Equivalent rate
RUBIK Pi 3	$3.0 + 2.8\,\mu$s	173 000 pkt/s
Arduino UNO Q	$18.2 + 15.2\,\mu$s	30 000 pkt/s

TABLE VI
MINIMUM TTL FOR FULL DELIVERY EQUALS HOP DIAMETER (WU–LI, SHIPPED FORWARDER, GEOMETRIC RADIO).

n	Degree	Diam. (hops)	Min TTL	Deliv. @ TTL 10
10	3.9	4	4	100%
20	4.3	8	8	100%
30	3.3	11	11	93%
50	5.2	13	13	94%
100	5.2	19	19	23%

a) Shared anti-replay window.: Hello/ANNOUNCE and DATA were numbered from independent counters but checked against one window. A DATA frame whose sequence equalled an earlier ANNOUNCE from the same node was dropped by every receiver and every relay. Reproduction: sequence 4 delivered 5/5, sequence 5 (the ANNOUNCE) 0/5, sequence 6 delivered 5/5. Likely cause of the 2/429 soak losses. Fix: one window per counter.

b) Split-brain coordinator.: Election used Hello-resolved neighbours only; validation of a rekey also counted ANNOUNCE-learned peers. On a stringy 30-node graph (mean degree 3.3) four nodes declared themselves coordinator; revocation was refused mesh-wide while the initiator's epoch advanced; 28 of 29 legitimate members stayed on the old key. Fix: one membership view for both.

E. What the six-node stand measured

The stand in front of us is no longer Figure 1. A third RUBIK Pi was added, same rule: peripheral, never dials. Each of the three screens held three links and each of the three UNO Q relays held three, which is $K_{3,3}$ (nine edges). The tables above stay what they are: they were not collected on this graph. The following was.

Fifteen group messages, sent from one screen a few seconds apart, were received by all five other boards: 75 receptions out of 75. Round to round, the arrival time on a given board moved by less than a tenth of a second. The boards' clocks are not aligned tightly enough, over SSH, to turn that into a millisecond one-way delay, so none is claimed.

Revocation matches Section III-E on this hardware. After one screen was excluded it did not display the next message; the other four boards did. Resetting the demonstration put all five back on the bootstrap key, and the next message reached all five. Excluding one relay left both screens able to read the message. The excluded relay did not.

Adding a node is dominated by the boot, not by the mesh. Rebooting the third screen, the first group message it could decrypt arrived 142 s after the reboot command and 4.4 s after its mesh daemon was running. Cutting a UNO Q controller loose on purpose, that relay received a group message again 130 s later, without a reboot of the board.

What this geometry still cannot answer from the bench is range. Every board sits in the same room, so a two-hop path is a choice of role, not a proof that the screens are out of range of each other. A corridor placement (screens at the ends, relays between, screens unable to dial one another) and a walk out of range and back are the remaining physical trials. A full-day delivery run is the remaining unattended one.

Until the corridor trial and the day-long run exist, a hop count on the screen is evidence of forwarding, not of radio range, and 75 receptions are a short trial, not a day of demonstration.

VI. CONCLUSION

An open BLE mesh with private groups is implementable without pairing. On the boards we have, the mesh converges in seconds, the CDS does not stretch paths, and the default TTL is a hop budget rather than a magic node count. Revocation works once keys have been announced and fails in a documented window if they have not. Cryptography is not the throughput limit; the Arduino host stack is. Keeping that host stack up for a day further requires the link discipline of Section III-G: the UNO Q must be central, its CoCs must be recycled before the signalling-identifier pool runs out, and a dead controller must be recovered by the board that holds it.

The stand of Section IV, since extended to three screens, is what we can put in front of an audience. Section V-E is the first timed pass on that graph: delivery, revocation, boot-to-group, and recovery from a dead controller. Indoor range, a walk out and back, and a full day of delivery remain open. Energy of AES-GCM versus the radio, and an Android CoC port, are future work.

ACKNOWLEDGMENT

The protocol note [2] acknowledges Matthieu Finiasz (OLVID) and Thomas Jaussouin and Olivier Berni (Lunabee); that debt carries over.

REFERENCES

[1] *Mesh Profile Specification*, Bluetooth SIG.

[2] N. Achir and P. Jacquet, "BUBBLE-BLUE a multihop private network based on Bluetooth," *arXiv preprint arXiv:2509.00967*, 2025.

[3] *Bluetooth Core Specification*, Bluetooth SIG, 2019, v5.0 / v5.1, LE L2CAP Connection-Oriented Channels.

[4] J. Wu and H. Li, "On calculating connected dominating set for efficient routing in ad hoc wireless networks," in *Proc. 3rd Int. Workshop on Discrete Algorithms and Methods for Mobile Computing and Communications (DIAL-M)*, 1999, pp. 7–14.

[5] ——, "A dominating-set-based routing scheme in ad hoc wireless networks," *Telecommunication Systems*, vol. 18, no. 1, pp. 13–36, 2001.

[6] A. Qayyum, L. Viennot, and A. Laouiti, "Multipoint relaying for flooding broadcast messages in mobile wireless networks," in *Proc. 35th Hawaii Int. Conf. on System Sciences (HICSS)*. IEEE, 2002, pp. 3866–3875.

[7] T. Clausen and P. Jacquet, "Optimized link state routing protocol (OLSR)," RFC 3626, 2003.

[8] P. Jacquet, "Analytical results on connected dominating sets in mobile ad hoc networks," INRIA, Tech. Rep., 2004.

[9] K. Kanchanasut, T. Wongsaardsakul, M. Chansutthirangkool, A. Laouiti, H. Tazaki, and K. R. Arefin, "DUMBO II: a V-2-I emergency network," in *Proc. 4th Asian Conf. on Internet Engineering*, 2008, pp. 37–38.

[10] J. Stoll, “BitChat,” https://github.com/permissionlesstech/bitchat, 2025.
[11] The Briar Project, “Briar,” https://briarproject.org/, 2024.
[12] M. Blomnes, “Reticulum / LXMF,” https://reticulum.network/, 2024.
[13] Netcore Network, “Causeway,” 2026, heterogeneous peer-to-peer networking.
[14] M. Dworkin, “Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC,” NIST SP 800-38D, 2007.
[15] J. Daemen and V. Rijmen, *The Design of Rijndael: AES — The Advanced Encryption Standard*. Springer, 2002.
[16] A. Langley, M. Hamburg, and S. Turner, “Elliptic curves for security,” RFC 7748, 2016.
[17] S. Josefsson and I. Liusvaara, “Edwards-Curve digital signature algorithm (EdDSA),” RFC 8032, 2017.